Oracle SQL Notes: Stored Procedures

Beginner-friendly notes • Examples • Syntax • Practice

1. What is a Stored Procedure?

A **stored procedure** is a named block of SQL/PL/SQL code stored in the database that you can execute whenever required. It helps you reuse common operations instead of rewriting the same SQL statements.

Example idea: instead of writing an INSERT query every time you add a course, create a procedure called `register_course()` and call it whenever needed.

2. Basic Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(
    parameter_name IN datatype
)
AS
BEGIN
    -- SQL statements
END;
/
```

Keyword meanings

Keyword	Purpose
CREATE OR REPLACE	Creates a procedure or replaces an existing one.
PROCEDURE	Defines a stored procedure.
IN	Passes a value into the procedure.
AS	Starts the declaration section.
BEGIN	Starts the executable section.
END	Marks the end of the procedure.
/	Runs the completed PL/SQL definition in clients that support this delimiter.

3. Example: Register a New Course

Step 1 — Create a table

```
CREATE TABLE courses (
    course_id NUMBER PRIMARY KEY,
    course_name VARCHAR2(100),
    fee NUMBER
);
```

Step 2 — Create a procedure

```
CREATE OR REPLACE PROCEDURE register_course (
    p_id IN NUMBER,
    p_name IN VARCHAR2,
    p_fee IN NUMBER
)
AS
BEGIN
    INSERT INTO courses
    VALUES (p_id, p_name, p_fee);

    DBMS_OUTPUT.PUT_LINE('Course Registered');
END;
/
```

Step 3 — Call the procedure

```
BEGIN
    register_course(201, 'Python Programming', 5000);
END;
/
```

Step 4 — Verify the result

```
SELECT *
FROM courses;
```

Expected row: `201 | Python Programming | 5000`

4. Passing Parameters to Procedures

Parameters let you send values into a procedure. This example updates the fee for an existing course.

Create procedure

```
CREATE OR REPLACE PROCEDURE update_course_fee (
    p_id IN NUMBER,
    p_new_fee IN NUMBER
)
AS
BEGIN
    UPDATE courses
    SET fee = p_new_fee
    WHERE course_id = p_id;
END;
/
```

Call it

```
BEGIN
    update_course_fee(201, 6500);
END;
/
```

Verify

```
SELECT *
FROM courses
WHERE course_id = 201;
```

The fee changes from 5000 to 6500.

5. Common Operations Using Procedures

INSERT — Add a record

```
INSERT INTO courses
VALUES (202, 'SQL Basics', 3000);
```

Adds a new row.

UPDATE — Modify a record

```
UPDATE courses
SET fee = 4000
WHERE course_id = 202;
```

Changes values in existing rows.

DELETE — Remove a record

```
DELETE FROM courses
WHERE course_id = 202;
```

Removes matching rows.

These are standalone SQL examples. You can place each statement inside a procedure's BEGIN ... END section.

6. Procedure vs Function

Feature	Procedure	Function
Main purpose	Performs an operation.	Computes and returns a value.
Return value	No required return value.	Must declare a return type.
Invocation	Commonly called using BEGIN ... END.	Can often be used in a SELECT.
Example	register_course()	get_course_fee()

Remember: procedures can use OUT parameters to return values, even though they do not have a mandatory RETURN clause like functions.

7. Important Points to Remember

• A procedure is stored in the database and can be reused.

• IN parameters supply input values.

• INSERT, UPDATE, and DELETE can be executed inside procedures.

• DBMS_OUTPUT.PUT_LINE() displays messages when output is enabled and supported by your SQL client.

• COMMIT saves a transaction. It is not mandatory in every procedure; transaction control depends on the application's requirements.

• Always verify the result with a SELECT query.